New Horizons®
Centro de Capacitación

Curso

Desarrollo Full Stack con C# y Angular

Docente: Edwin Barrientos



New Horizons®
Centro de Capacitación

Sesión 12 — Módulo 3:

Pantallas del Flujo de Aprobación



Objetivo del módulo: completar el flujo de negocio interactivo de solicitudes de dinero en Angular 18, implementando bandejas por rol organizativo, modales de dictamen con MatDialog y la visualización de trazabilidad y auditoría mediante el componente Timeline.



New Horizons®
Centro de Capacitación

Agenda

- Arquitectura de flujo de aprobación: Estados, roles y reglas de transición
- Bandejas de trabajo por rol: Solicitante, Revisor, Aprobador y Tesorería
- Endpoints de transición: Enviar, Revisar, Aprobar, Rechazar y Pagar
- Modales de dictamen: Captura de comentarios y datos de pago con MatDialog
- Componente Timeline: Auditoría histórica y trazabilidad end-to-end



New Horizons®
Centro de Capacitación

Sesión 1:

Pantallas y Modales del Flujo de Aprobación End-to-End

El flujo corporativo completo

New Horizons
Centro de Capacitación

Como un trámite con sellos en ventanilla. La solicitud nace en Borrador, pasa por Revisión, recibe la firma de Gerencia según el monto y termina pagada en Tesorería. Cada ventanilla deja su sello: nadie puede saltarse una.

Roles Múltiples

Solicitante, Revisor técnico, Aprobador gerencial y Cajero de Tesorería.

Piénsalo así: *Cada ventanilla tiene su propio funcionario y solo atiende su parte del trámite.*

Máquina de Estados

Transiciones gobernadas por la entidad Solicitud en el dominio de C#.

Piénsalo así: *Es el reglamento de la ventanilla: dice a qué paso puedes ir desde el paso actual.*

Auditoría Total

Cada dictamen deja huella inmutable con fecha, responsable y comentario.

Piénsalo así: *Es el sello fechado en el expediente: no se borra ni se reescribe.*

Diseño de bandejas por rol organizativo

New Horizons
Centro de Capacitación

📬 **Casilleros de una oficina:** cada colaborador abre solo su casillero y encuentra únicamente los documentos que le toca atender. Nada de ruido ajeno.

📝 Bandeja Solicitante

Filtro

`Borrador · todas las mías`

Mis solicitudes, borradores pendientes de envío y estados de aprobación.

🔍 Bandeja Revisor

Filtro

`estado = Enviada`

Solicitudes pendientes de verificación técnica o presupuestal.

✍️ Bandeja Aprobador

Filtro

`estado = EnRevision`

Listas para firma gerencial con validación de límites.

🏦 Bandeja Tesorería

Filtro

`estado = Aprobada`

Listas para desembolso, pago y conciliación bancaria.

Simulación de roles con RolService

New Horizons
Centro de Capacitación

rol.service.ts

```ts
import { Injectable, signal } from "@angular/core";

export enum RolUsuario { Solicitante, Revisor, Aprobador, Tesoreria }

@Injectable({ providedIn: "root" })
export class RolService {
  usuarioActual = signal({
    id: 1, nombre: "Juan Pérez", rol: RolUsuario.Solicitante
  });

  cambiarRol(nuevo: RolUsuario): void {
    // En la Sesión 13-14 se vinculará con los Claims del Token JWT
    this.usuarioActual.update(u => ({ ...u, rol: nuevo }));
  }
}
```

El camerino del teatro

Un mismo actor cambia de vestuario (rol) para cada escena, hasta que llegue el DNI real: el JWT.

1 enum RolUsuario = el elenco

Los cuatro personajes posibles del flujo.

2 providedIn: "root" = un solo camerino

Singleton: toda la app ve al mismo usuario.

3 signal() = el actor en escena

Quien lo lea se entera al instante del cambio.

4 update() = cambio de vestuario

Crea un objeto nuevo con `...u`, sin mutar.

Botones condicionales según estado y rol

New Horizons®
Centro de Capacitación

botones-contextuales.html

```
<!-- Enviar a Revisión: solo Solicitante en Borrador -->
@if (solicitud.estado === 1 && esSolicitante()) {
  <button mat-raised-button color="primary" (click)="enviar()">
    <mat-icon>send</mat-icon> Enviar a Revisión
  </button>
}

<!-- Aprobar / Rechazar: solo Aprobador en EnRevision -->
@if (solicitud.estado === 3 && esAprobador()) {
  <button mat-raised-button color="primary"
          (click)="abrirModalAprobar()">Aprobar</button>
  <button mat-stroked-button color="warn"
          (click)="abrirModalRechazar()">Rechazar</button>
}
```

1
2



Puerta con lector de tarjeta

Se abre solo con **dos condiciones**: tu tarjeta (rol) y el horario correcto (estado).

¿Qué botón aparece?

Borrador + Solicitante	**Enviar**
EnRevision + Aprobador	**Aprobar / Rechazar**
Cualquier otra combinación	**Ninguno**

⚠️ Los números `1` y `3` son “mágicos”. Mejor un enum: `EstadoSolicitud.Borrador`. Y recuerda: ocultar el botón es UX, **no seguridad**.

Endpoints de transición en C#

New Horizons
Centro de Capacitación



Una línea de metro: cada endpoint es una estación y desde cada parada solo puedes ir a la siguiente. Rechazar es la salida de emergencia: el viaje termina ahí.

POST	`/api/solicitudes/{id}/enviar`	Borrador	→	Enviada	Valida integridad del documento.
POST	`/api/solicitudes/{id}/revisar`	Enviada	→	EnRevision	Asigna el revisor técnico.
POST	`/api/solicitudes/{id}/aprobar`	EnRevision	→	Aprobada	Comprueba el tope presupuestal.
POST	`/api/solicitudes/{id}/pagar`	Aprobada	→	Pagada	Registra el movimiento en Tesorería.
POST	`/api/solicitudes/{id}/rechazar`	En curso	→	Rechazada	Exige motivo · estado terminal.

Llamadas a transiciones en SolicitudHttpService

New Horizons®
Centro de Capacitación

transiciones-service.ts

```
enviar(id: number): Observable<Solicitud> {
  return this.http.post<Solicitud>(
    `${this.baseUrl}/${id}/enviar`, {});
}

aprobar(id: number, comentario: string,
        respId: number): Observable<Solicitud> {
  return this.http.post<Solicitud>(`${this.baseUrl}/${id}/aprobar`,
    { comentario, responsableId: respId });
}

rechazar(id: number, motivo: string,
         respId: number): Observable<Solicitud> {
  return this.http.post<Solicitud>(`${this.baseUrl}/${id}/rechazar`,
    { motivo, responsableId: respId });
}
```



El mensajero con sobres

Cada método es un sobre distinto: misma oficina, otra ventanilla y otro contenido.

1 La dirección del sobre

La URL lleva el `id` y la transición: `/42/aprobar`.

2 Observable = acuse de recibo

La respuesta llega después: la solicitud ya actualizada.

3 El contenido del sobre

`enviar` va vacío `{}`; aprobar y rechazar llevan comentario o motivo y responsable.

Modal DictamenDialogComponent

New Horizons®
Centro de Capacitación

abrir-dictamen.ts

```
abrirModalAprobar(): void {
  const ref = this.dialog.open(DictamenDialogComponent, {
    data: { tipo: "aprobar", codigo: this.solicitud.codigo }
  });
  ref.afterClosed().subscribe(comentario => {
    if (comentario) {
      this.httpService
        .aprobar(this.solicitud.id, comentario, 3)
        .subscribe(actualizada => {
          this.solicitud = actualizada;
          this.cargarTimeline(); // Refresca auditoría
        });
    }
  });
}
```



La cabina de votación

Abres la cabina, el votante deja su voto en privado y **solo al salir** cuentas el voto.

1 open() = abrir la cabina

Le pasas `data`: el tipo de dictamen y el código.

2 afterClosed() = salir de la cabina

Emite lo que el modal devolvió con `close(valor)`.

3 if (comentario) = voto válido

Esc o clic afuera devuelven `undefined`: no se envía nada.

4 cargarTimeline() = actualizar el acta

La auditoría se refresca tras el éxito.

Modal PagoDialogComponent (Tesorería)

New Horizons
Centro de Capacitación

Sin voucher no hay pago. Igual que en la caja de un banco, el diálogo de Tesorería no cierra el trámite hasta registrar el medio de pago y el número de operación.

Registrar pago · SOL-00004

Medio de pago

Transferencia BCP ▾

N° de operación

00012345678

Cancelar

💰 Registrar pago

Medio de Pago

Desplegable MatSelect con bancos o fondos fijos.

N° Operación

Identificador único de la transferencia para la conciliación contable.

Estado Final

Transiciona a Pagada y genera registro en MovimientoTesoreria.

Componente Timeline: Trazabilidad visual

New Horizons
Centro de Capacitación

timeline.component.html

```
<!-- Línea de tiempo que renderiza TimelineItem[] -->
<div class="timeline-list">
  @for (item of items; track item.id; let last = $last) {
    <div class="timeline-item">
      <div class="badge">
        <mat-icon>{{ obtenerIcono(item.estado) }}</mat-icon>
      </div>
      <div class="contenido">
        <strong>{{ item.estadoNombre }}</strong> •
        <span>{{ item.fechaAccion | date:'short' }}</span>
        <p>Por: {{ item.responsableNombre }}</p>
        @if (item.comentario) {
          <blockquote>"{{ item.comentario }}"</blockquote>
        }
      </div>
    </div>
  }
</div>
```

1. Línea de tiempo que renderiza TimelineItem[]
2. Cada entrada: qué pasó, cuándo, quién firmó y una nota opcional.
3. Enviada 10/10 · Juan Pérez
4. En Revisión 11/10 · Ana Torres

La bitácora del capitán

Cada entrada: qué pasó, cuándo, quién firmó y una nota opcional.

Enviada

10/10 · Juan Pérez

En Revisión

11/10 · Ana Torres

Aprobada

12/10 · Gerente Finanzas

“Conforme”

Consumo del endpoint GET /api/solicitudes/{id}/timeline

New Horizons®
Centro de Capacitación

cargar-timeline.ts

```
cargarTimeline(): void {
  this.httpService.obtenerTimeline(this.solicitud.id).subscribe({
    next: (eventos) => {
      this.eventosTimeline = eventos;
    },
    error: (err) =>
      console.error("Error al obtener trazabilidad:", err)
  });
}
```

1 GET /api/solicitudes/{id}/timeline

2 next = llegó el expediente

3 error = aviso del archivo

Pedir el expediente al archivo

Haces el pedido, esperas, y te llega el expediente completo… o una nota de que no se encontró.

1 GET /api/solicitudes/{id}/timeline

El pedido al archivo central.

2 next = llegó el expediente

Guardamos los eventos y el template se repinta.

3 error = aviso del archivo

Lo registramos; el ErrorInterceptor avisa al usuario.

¿Cuándo se llama a cargarTimeline()?

Al abrir el detalle

`ngOnInit`

Tras aprobar o rechazar

`afterClosed → OK`

Tras registrar el pago

`POST /pagar → OK`

Concurrencia y consistencia en el Servidor

New Horizons
Centro de Capacitación

El último asiento del avión: dos personas lo compran en el mismo segundo; solo el sistema central decide quién se lo lleva. El backend es la máxima autoridad aunque el frontend oculte botones.

Gerente A	`POST /42/aprobar` →	**200 OK · Aprobada**	Llega primero: la transición es válida.
Gerente B	`POST /42/aprobar` →	**400 · ReglaDominioException**	Llega 1 s después: la solicitud ya no está EnRevision.

ReglaDominioException

Lanzada si se intenta aprobar una solicitud ya aprobada o pagada.

Feedback en UI

El ErrorInterceptor captura el fallo e informa al usuario al instante.

Refresco Automático

La vista consulta nuevamente el estado actual del documento en el servidor.

El ciclo de vida completo de la Solicitud

New Horizons
Centro de Capacitación

Como el rastreo de un paquete por courier: siempre sabes en qué tramo está y quién lo tiene en ese momento.

1 Borrador

📦 *Paquete armado*

- Creada por usuario
- Partidas de gasto

2 Enviada

📦 *Recogido*

- Enviada formalmente
- Espera a Revisor

3 En Revisión

📦 *En tránsito*

- Evaluación técnica
- Firma gerencial

4 Aprobada

📦 *En reparto*

- Gerencia conforme
- Pasa a Tesorería

5 Pagada

📦 *Entregado*

- Desembolso bancario
- Estado final

🛑 **¿Y Rechazada?** Es la vía muerta: se puede llegar desde la revisión o la aprobación y el viaje termina ahí. Lo vemos en la siguiente lámina.

Manejo de caminos alternativos: Rechazo

New Horizons
Centro de Capacitación



Un cambio de vías: aprobar mantiene el tren en la línea principal; rechazar lo desvía a la vía muerta, donde el viaje termina.

✅ Flujo de Aprobación

- ✓ Avanza hacia la siguiente instancia de control
- ✓ Comentario de visto bueno opcional pero recomendable
- ✓ Mantiene la solicitud viva en el proceso contable

⇆

⛔ Flujo de Rechazo

- ✗ Finaliza inmediatamente el ciclo del documento
- ✗ Obliga al aprobador a ingresar motivo de rechazo
- ✗ Notifica al solicitante en el timeline para corrección

Estructura de la pantalla de Detalle

New Horizons
Centro de Capacitación

Como la portada de un periódico: titular arriba, la noticia al centro y la historia al costado. El usuario entiende el documento en cinco segundos.



Cabecera = el titular

Código, motivo, solicitante, centro de costo y badge de estado.

Desglose = la noticia

Tabla con conceptos, cantidades, precios unitarios y total.

Timeline = la historia

Cronología vertical con todas las firmas y dictámenes.

Checklist de demostración en vivo

New Horizons
Centro de Capacitación

Una obra en cinco actos: el mismo actor cambia de vestuario con el RolService y la solicitud SOL-00004 recorre todo el escenario.

1	Acto 1	Solicitante	Crea SOL-00004 y la envía a revisión.
2	Acto 2	Revisor	Inicia la evaluación cambiando a EnRevision.
3	Acto 3	Gerente	Aprueba en el modal con el comentario 'Conforme'.
4	Acto 4	Tesorería	Registra el pago vía Transferencia Bancaria BCP.
5	Acto 5	Todos	Visualizan los 4 hitos en el Timeline con sus responsables.

Valor para la empresa: Trazabilidad y Compliance



La cámara de seguridad del negocio: si algún día alguien pregunta “¿quién aprobó este gasto?”, la respuesta está grabada con fecha y nombre.



Cero Anonimato

Cada cambio de estado queda registrado con el ID del responsable.

Antes: “No sé quién lo firmó”.

Ahora: “Gerente Finanzas, 12/10 10:32”.

Sin Papel

Sustituye formatos físicos y firmas manuales por un flujo digital.

Antes: el formato se perdía en un escritorio.

Ahora: el trámite avanza solo, sin mensajeros.



Información al Día

Finanzas y Contabilidad conocen la deuda comprometida en tiempo real.

Antes: cierre a fin de mes.

Ahora: el compromiso se ve al instante.

Checklist de calidad del flujo de aprobación

New Horizons®
Centro de Capacitación



La revisión técnica vehicular: cinco puntos de inspección antes de recibir la calcomanía de “apto para producción”.

✓	**Acciones de transición protegidas según el rol.**	→ *Visual en Angular y operativa en C#.*
✓	**Diálogos MatDialog con justificaciones obligatorias.**	→ *Nadie rechaza sin explicar por qué.*
✓	**Timeline consumiendo el historial del backend.**	→ *Siempre refleja la verdad del servidor.*
✓	**KPI del Dashboard actualizados de forma reactiva.**	→ *Los números cambian al aprobar o pagar.*
✓	**Persistencia auditada en ASP.NET Core.**	→ *Cada transición queda guardada en la BD.*

Errores comunes en flujos de negocio

New Horizons
Centro de Capacitación

✗ Evita esto	✓ Haz esto
Confiar en el frontend para validar permisos o montos	Deja que el backend valide siempre la máquina de estados
Permitir rechazar sin motivo explicativo	Haz obligatorio el motivo de rechazo en el MatDialog
No refrescar el timeline tras una aprobación	Llama a cargarTimeline() tras cada respuesta positiva
Editar o borrar solicitudes ya enviadas o pagadas	Bloquea la edición si el estado no es Borrador
Perder los datos bancarios al cerrar el modal	Envía medio y N° de operación en el body de POST /pagar

Hacia la Sesión 13: Seguridad y JWT

Del gafete de visitante al fotocheck con chip: hoy el rol se elige con un clic (simulación); en la Sesión 13 lo certificará un token JWT emitido por ASP.NET Core.

Hoy · Sesión 12

- Rol simulado con RolService
- Botones ocultos según el rol
- Backend sin [Authorize]

✓ **Pantallas y modales ya listos**

Próxima · Sesión 13

✓ Rol leído de los claims del JWT

✓ POST /auth/login en C#

✓ Controladores con [Authorize]

✓ HttpInterceptor que adjunta el Bearer token

Resumen de la sesión

New Horizons®
Centro de Capacitación

👤 Bandejas Multirrol

Vistas adaptadas a solicitante, revisor y aprobador.

≈ Los casilleros

🔄 Transiciones

Endpoints /enviar, /revisar, /aprobar, /pagar.

≈ Las estaciones de metro

💬 Modales MatDialog

Dictamen con comentarios y medio de pago.

≈ La cabina de votación

🕐 Timeline

Visualizador histórico consumiendo la API de C#.

≈ La bitácora del capitán

🎯 Máquina de Estados

Consistencia absoluta entre cliente y servidor.

≈ El último asiento del avión

🎉 Flujo Completo

Aplicación Full Stack 100% operativa.

≈ El paquete entregado

PREGUNTAS...

New Horizons®
Centro de Capacitación

1. ¿Cómo garantizamos que un usuario solo pueda ejecutar las acciones permitidas para su rol?
2. ¿Cómo se conectan los botones de la UI con los endpoints POST /api/solicitudes/{id}/transicion?
3. ¿Por qué es obligatorio exigir un comentario al rechazar una solicitud en el modal?
4. ¿Cómo consumimos el endpoint GET /api/solicitudes/{id}/timeline para pintar la cronología?
5. ¿Qué ocurre si dos aprobadores intentan procesar la misma solicitud al mismo tiempo?

New Horizons
Centro de Capacitación

Conclusiones

Cada quien su casillero

La separación por roles en la UI ofrece una experiencia clara y sin saturación.

Botones con lector de tarjeta

Los botones contextuales reflejan en tiempo real las restricciones de la máquina de estados de C#.

Nada se envía sin justificar

Los modales garantizan la captura de justificaciones y datos de pago antes de enviar la petición.

Bitácora transparente

El componente Timeline aporta trazabilidad absoluta a auditoría y finanzas.

Paquete entregado

El ciclo de vida queda cerrado y operativo end-to-end entre Angular y ASP.NET Core.

GRACIAS